

UNIDADES TECNOLÓGICAS DE SANTANDER

Facultad de Ciencias Naturales e Ingenierías
Tecnología en Desarrollo de Sistemas Informáticos

INFORME TÉCNICO: DESARROLLO DE APLICACIÓN WEB INTERACTIVA CON CONSUMO DE DISNEY API

*Implementación de Paginación, Búsqueda Dinámica y Filtrado Client-Side con HTML5,
CSS3 y JavaScript Asíncrono*

Estudiante: Laura Marcela Calderon Rodriguez

Asignatura: Desarrollo Web

Docente: Profesor de Desarrollo Web

Institución: Unidades Tecnológicas de Santander (UTS)

Bucaramanga, Santander
2026

1. Introducción

En el desarrollo web moderno, la capacidad de interactuar de forma asíncrona con servicios externos e integrar información en tiempo real es una competencia fundamental. El modelo cliente-servidor se apoya en gran medida en las Arquitecturas de Transferencia de Estado Representacional (REST API) para intercambiar datos estructurados en formato JSON.

El presente documento expone el diseño, desarrollo y documentación de una aplicación web cliente (*Frontend*) orientada al consumo de la **Disney API** pública (`https://api.disneyapi.dev``). La aplicación permite explorar de manera dinámica un catálogo de miles de personajes del universo de Disney, garantizando una experiencia de usuario fluida mediante el uso de JavaScript nativo (ECMAScript 6+), manipulación del Modelo de Objetos del Documento (DOM), control de eventos y la Fetch API.

Se han priorizado las buenas prácticas de desarrollo web, implementando un diseño limpio, responsivo y adaptable a dispositivos móviles sin recurrir a bibliotecas excesivas ni estilos artificiales complejos, enfocándose en la eficiencia algorítmica de la paginación, la búsqueda asíncrona y el filtrado en el cliente.

2. Explicación de la API Consultada (Disney API)

La **Disney API** es un servicio web RESTful gratuito y de acceso público que consolida datos sobre personajes, películas, series de televisión, atracciones de parques temáticos y videojuegos producidos por The Walt Disney Company.

2.1. Estructura y Endpoints Principales

La API no requiere autenticación previa mediante llaves (API Keys), lo que facilita su integración directa desde aplicaciones cliente. El punto de entrada base para consultar personajes es:

```
GET https://api.disneyapi.dev/character
```

2.2. Esquema de Respuesta JSON y Paginación

Al realizar una petición `GET``, la API responde con un objeto JSON estandarizado estructurado en dos secciones clave: `info`` (metadatos de paginación) y `data`` (arreglo de objetos de personajes).

```

{
  "info": {
    "count": 7438,
    "totalPages": 149,
    "previousPage": "https://api.disneyapi.dev/character?page=1&pageSize=50",
    "nextPage": "https://api.disneyapi.dev/character?page=3&pageSize=50"
  },
  "data": [
    {
      "_id": 112,
      "name": "Achilles",
      "imageUrl": "https://static.wikia.nocookie.net/disney/images/d/d3/Achilles.png",
      "films": ["Hercules"],
      "tvShows": ["Hercules (tv series)"],
      "videoGames": ["Kingdom Hearts III"],
      "parkAttractions": []
    }
  ]
}

```

2.3. Parámetros de Consulta (Query Parameters)

La API admite parámetros de consulta URL para controlar la paginación y la búsqueda de recursos:

- **page** : Especifica el número de página solicitado (por defecto es 1).
- **pageSize** : Determina la cantidad de registros devueltos por página (por defecto es 50).
- **name** : Filtra personajes cuyo nombre coincida o contenga el término especificado (ej. **?name=Mickey**).

3. Explicación de la Arquitectura de la Aplicación Web

La arquitectura de la aplicación sigue el patrón estándar de una Aplicación de Página Única (SPA) cliente tradicional, construida sobre un enfoque modular sin dependencias externas pesadas.

Capas / Componente	Tecnología Utilizada	Descripción de Responsabilidad
Estructura (DOM)	HTML5 Semántico	Define la estructura de controles (input de búsqueda, select de filtro, botones de navegación) y el contenedor de tarjetas (grid layout).
Presentación y Layout	CSS3 (Flexbox & Grid)	Garantiza la adaptabilidad *responsive* en resoluciones móviles, tablets y escritorio mediante media queries y reglas fluidas.
Lógica de Negocio	JavaScript (ES6 Vanilla)	Maneja las peticiones HTTP asíncronas (fetch), el estado de la paginación, los filtros dinámicos y la renderización en el DOM.

3.1. Flujo de Datos y Manejo de Estado

La aplicación mantiene variables de estado global para controlar la URL activa, la colección de personajes cargados en la página actual y las opciones de filtro seleccionadas:

```
// Variables de control de estado
let urlActual = "https://api.disneyapi.dev/character?pageSize=20";
let personajesCargados = []; // Colección de personajes de la página activa

// Petición Asíncrona con Fetch API
function obtenerPersonajes(url) {
  contenedor.innerHTML = '<p class="cargando">Cargando personajes...</p>';

  fetch(url)
    .then(respuesta => {
      if (!respuesta.ok) throw new Error("Error en la solicitud HTTP");
      return respuesta.json();
    })
    .then(res => {
      // Manejo flexible según si la API retorna arreglo único o paginado
      personajesCargados = Array.isArray(res.data) ? res.data : [res.data];
      renderizarPersonajes(personajesCargados);
      actualizarPaginacion(res.info);
    })
    .catch(err => {
      contenedor.innerHTML = '<p class="error">No se encontraron resultados.</p>';
    });
}
```

3.2. Implementación de Búsqueda y Filtrado Dinámico

Para cumplir con los requerimientos técnicos del proyecto, la aplicación combina la búsqueda directa a través del parámetro `?name=` de la API con un filtro secundario en el lado del cliente (Client-Side Filtering) basado en las apariciones del personaje (Películas, Series o Videojuegos):

```
// Filtrado en tiempo real dentro de los datos cargados
function aplicarFiltroMedio(tipoMedio) {
  if (tipoMedio === "todos") {
    renderizarPersonajes(personajesCargados);
    return;
  }

  const filtrados = personajesCargados.filter(p => {
    if (tipoMedio === "films") return p.films && p.films.length > 0;
    if (tipoMedio === "tvShows") return p.tvShows && p.tvShows.length > 0;
    if (tipoMedio === "videoGames") return p.videoGames && p.videoGames.length > 0;
    return true;
  });

  renderizarPersonajes(filtrados);
}
```

4. Capturas de Pantalla y Funcionamiento de la Aplicación

A continuación se ilustra el comportamiento visual y la interfaz funcional de la aplicación web desarrollada en diferentes escenarios de uso y dispositivos.

4.1. Vista General del Catálogo y Paginación (Escritorio)

Muestra la interfaz principal cargando el primer bloque de personajes en un diseño Grid responsive con tarjetas limpias, visualización de apariciones destacadas y botones de paginación habilitados.

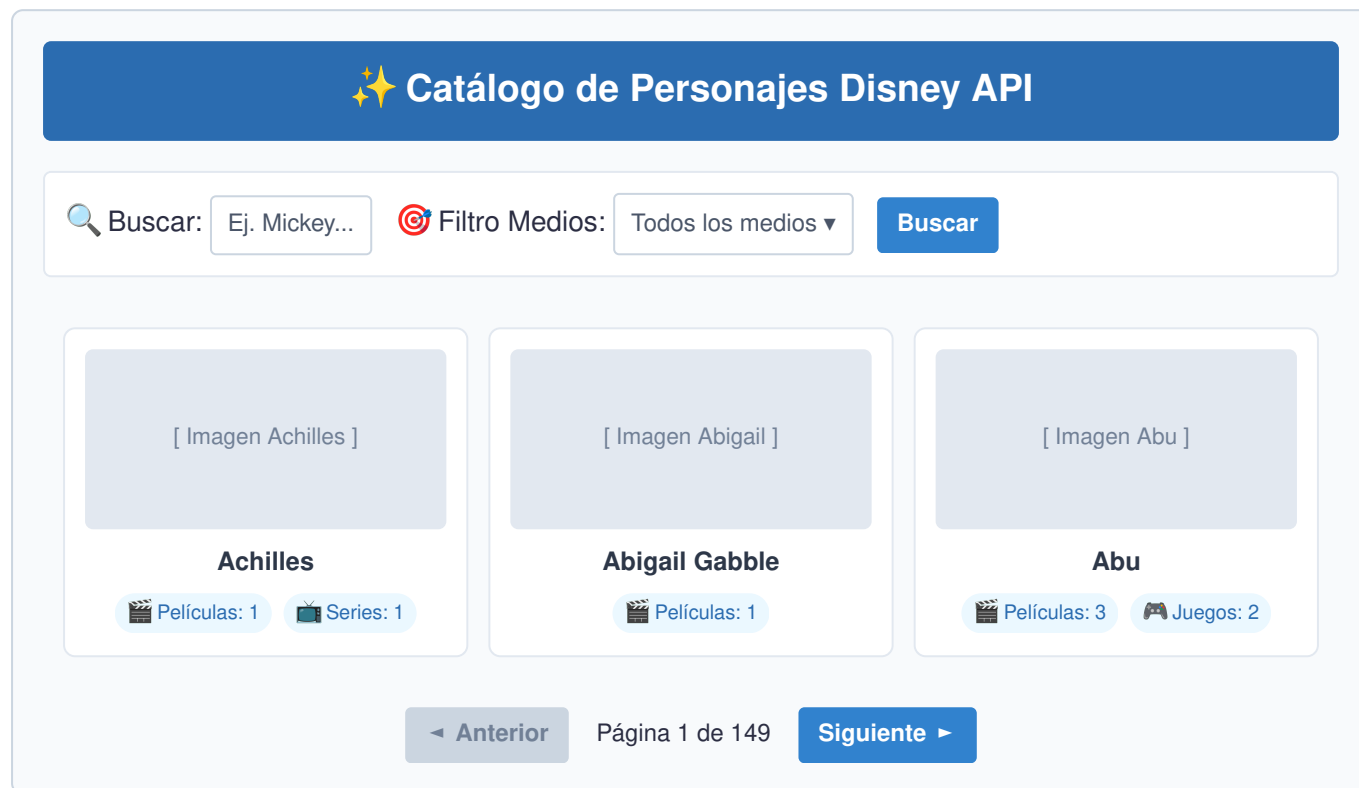


Figura 1. Vista de catálogo con renderización de tarjetas y controles de paginación activos.

4.2. Búsqueda por Nombre y Filtrado por Categoría

Muestra el resultado al ingresar la consulta "Aladdin" en la barra de búsqueda y aplicar el filtro de categorías para mostrar únicamente personajes que han aparecido en videojuegos.



Figura 2. Resultados filtrados dinámicamente mediante eventos de entrada en el cliente.

4.3. Adaptabilidad Responsive en Dispositivos Móviles

La maquetación con CSS Media Queries reestructura la disposición horizontal de múltiples columnas en una sola columna vertical fluida para pantallas pequeñas (móviles).

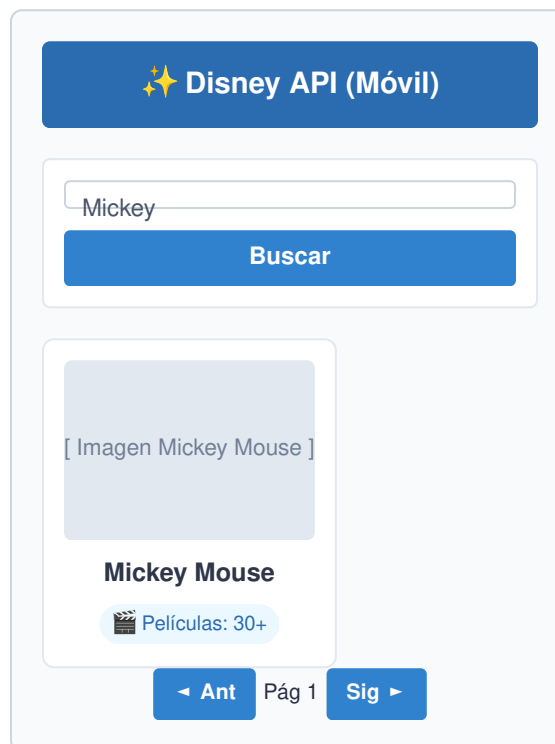


Figura 3. Disposición adaptada a pantalla móvil (Single Column Layout).

5. Conclusiones

- La utilización de la **Disney API** demostró ser una alternativa robusta e idéntica en comportamiento a la API de *Rick and Morty*, permitiendo poner en práctica el consumo de endpoints paginados y la estructuración de datos en formato JSON.
- El uso de JavaScript nativo (*Vanilla JS*) y la **Fetch API** permitió comprender a fondo la naturaleza asíncrona de las promesas en JavaScript, garantizando un control preciso sobre el estado del DOM sin depender de frameworks pesados.
- La implementación combinada de búsqueda basada en parámetros URL (Server-Side) con filtrado local de arreglos (Client-Side) ofrece una experiencia de usuario rápida e interactiva.
- Mantener un estilo visual simple y natural mediante reglas CSS semánticas y layout fluido (Grid/Flexbox) asegura una aplicación limpia, de carga ultrarrápida y completamente adaptable (*responsive*) a cualquier pantalla.

6. Referencias Bibliográficas

Disney API. (2026). *Disney API Documentation and Endpoint Specification*. Recuperado de <https://disneyapi.dev/>

Mozilla Developer Network (MDN). (2025). *Using the Fetch API*. MDN Web Docs. Recuperado de https://developer.mozilla.org/es/docs/Web/API/Fetch_API/Using_Fetch

Mozilla Developer Network (MDN). (2025). *CSS Grid Layout: Responsive Design*. MDN Web Docs. Recuperado de https://developer.mozilla.org/es/docs/Web/CSS/CSS_Grid_Layout

W3C. (2024). *HTML5 Accessibility and Responsive Web Architecture Guidelines*. World Wide Web Consortium.